Amber Tool Validation

Department Name

June 8, 2026

Abstract

This document records validation of Amber as the Configuration Item identified in this Report Package. It describes the intended use requirements, execution evidence, deviations, and review conclusion used to determine whether the tool is fit for its intended use in the controlled software lifecycle. After the Report Package has been executed and the evidence supports the conclusion that the applicable Intended Use Requirements (IUR) are satisfied, Amber is retained in the Amce Corporation (COMPANY) Quality Management System.

Contents

Approval Signatures

Role	Name	Signature
Author	Gary A. Howard	Electronic signature on file.
Project Lead	N/A	Electronic signature on file.
Technical Lead	N/A	Electronic signature on file.
System Engineer	N/A	Electronic signature on file.
Software Engineer	N/A	Electronic signature on file.
Test Engineer	N/A	Electronic signature on file.
Product Manager	N/A	Electronic signature on file.
Clinical App. Manager	N/A	Electronic signature on file.
RA Specialist	N/A	Electronic signature on file.
QA Engineer	Dayn A. Howard	Electronic signature on file.
Medical Doctor	N/A	Electronic signature on file.
Technical Reviewer	Cj Howard	Electronic signature on file.
Technical Reviewer	N/A	Electronic signature on file.
Technical Reviewer	N/A	Electronic signature on file.
Technical Reviewer	N/A	Electronic signature on file.
Independent Reviewer	N/A	Signature attached.

1 Introduction

1.1 Overview

This report demonstrates the Amber driver consumes YAML Test Plans, Test Suites, and Test Cases and produces output that is properly formatted for reports designed to tlc-article and audotoc conventions.

1.2 Purpose

This Report Package provides the controlled record for validation of the identified Configuration Item as a software lifecycle tool. It identifies the tool's Intended Use Requirements (IUR), the test reports executed, the configuration used for execution, and the objective evidence generated by each validation activity.

The record supports the validation conclusion by preserving the pass/fail result for each test step and test report, the disposition of deviations or anomalies, and confirmation that the Configuration Item has an approved Configuration Identification.

1.3 Scope

This Report Package applies to COMPANY medical device software projects that require validation of a Configuration Item for its intended use. It covers the activities, records, and conclusions used to determine whether the Configuration Item satisfies its applicable Intended Use Requirements (IUR).

The scope should identify the validated configuration, intended use, environment, required tools, input data, excluded uses, and limitations that affect interpretation of the validation conclusion.

1.4 Deviations

The process governing the creation of this protocol and report deviates from the normal standard operating procedure (SOP005 Validation of Computerized Systems). This document combines both the protocol and the report. Normally the protocol is released first, and the report is released after the protocol is executed. This document represents an automated protocol execution facilitated through the use of automation scripting and software. The review of a paper protocol and pre-approval of said protocol does not satisfy the need to review the automated components used for the generation of this document. As a result, the automated components which codify the actual test protocol are reviewed

by a technical approver as this document and the components are developed. This technical approver is an approver of this document and their approval indicates the automated components effectively test the article under test to meet the intended use as specified in the user requirements.

Additionally, data obtained from the execution of the protocol is collected and presented in the grey boxes as objective evidence from the automated test application. Normally this would not be presented together with the protocol, but given that this is an automated process in a combined document, this is an effective means of retaining and presenting the objective evidence for review and approval.

Finally, presenting the protocol and the report together allows for a single step automation process that can be easily maintained and re-executed. Re-execution is often desired due to changes to the article under test or changes to user needs.

1.5 Tool Validation Objectives

Document 123-VNV-056022 Validation Determination report provides a Determination of Validation decision tree and Determination of Level of Risk and Validation Rigor decision tree to aid a Development Team when assessing the need for validation. 123-VNV-056022 was updated to indicate this tool required Validation and the Level of Risk needed. The steps below describe the steps used to validate a tool.

1. Describe the intended use of the tool.
2. Set the purpose and scope for the tool validation effort.
3. Enumerate intended use requirements.
4. Disclose compliance criteria.
5. Define Tool validation acceptance criteria.
6. Identify responsible persons and their roles.
7. Document required deliverables.
8. Define specific test steps and test steps to confirm that the Tool's intended use requirements have been met.
9. Collect test evidence.
10. Record Tool validation conclusion.

1.6 General Terms

Configuration Control The systematic process for managing changes to and established baseline.

Configuration Identification A unique identifier used to associate a collection of software artifacts.

Configuration Items Software source code, executables, build scripts, and other software development and software test artifacts relevant to creating and maintaining a software project.

Configuration Status Accounting The recording and reporting of the information needed to effectively manage the software and documentation components of a software project.

Report Package A detailed record that provides a Configuration Item overview, a list of its Intended Use Requirements (IUR), one or more Test Reports, evidence the Test Reports ran along with the output produced by Test Report including a pass/fail result for each Test Step and Test Report, and a statement indicating the Configuration Item has a Configuration Identification, and conclusion that the Configuration Item has been validated for its intended use.

Test Plan A test plan is a collection of one or more test suites a tester has determined to use to challenge requirements.

Test Suite A test suite is a collection of one or more test cases a tester has determined to use to challenge requirements.

Test Case A test case is a set of conditions under which a tester will determine whether the test is working as it was originally established for it to do.

Test Step A unique test identifier with predetermined expectation, confirmation criteria, and pass/fail result.

Test Report A test report consists of Detailed instructions for the set-up, execution, and evaluation of results for a given test. The test protocol may include one or more test cases for which the steps of the protocol will repeat with different input data. Test cases are chosen to ensure that corner cases in the code and data structures are covered. A test protocol may be a script that is automatically run by the computer.

1.7 General Acronyms

FDA Food and Drug Administration

IUR Intended Use Requirements

LMS Learning Management System

SOP Standard Operating Procedure

SOUP Software Of Unknown Provenance

1.8 References

- SOP004 Software Devopment Procedure
- SOP003 Software Configuration Management
- SOP001 Good Documentation Practices
- SOP005 Validation of Computerized Systems
- SOP002 Change Management

1.9 Training

Company's training records are stored in the Quality Management System. Additional training is not required because this is an automated test that is executed by the automated testing platform. SOP004 Software Devopment Procedure provides training required to create, maintain, and execute this testing protocol.

1.10 Tool Validation Test Approach

This Test Plan describes a series of Test Suites, Test Cases, and Test Steps. When executed, each Test Step determines if the Configuration Item satisfies one or more system requirements. When a Test Step indicates that the system requirements are satisfied, the Test Step's result is "pass". Otherwise, the Test Step's result is "fail". The computer records all "pass" and "fail" results in the Test Plan record. The Configuration Item is considered verified when all Test Steps are executed and the Test Plan record contains no "fail" results. Each Test Step that results in a deviation, observation, incident, or failure shall be represented in the final report.

1.11 Configuration Management

When a Configuration Item is changed, we will review the manufacturer's release notes or our Design History File (DHF) (DHF) to determine if regression testing or adjustments to this Report Package are necessary. We will verify the changes do not impact product operation, product quality, or quality decisions made prior to performing the upgrade.

1.12 Test Plan Instructions

This Test Plan describes Test Suites, Test Cases, and Test Steps that demonstrate how the Configuration Item satisfies the IUR. Each Test Plan describes any setup criteria needed to conduct the test. Each Test Plan contains a list of IURs and the steps that demonstrate how the Configuration Item satisfies the IUR. Each Test Step is marked passed or failed as it is completed. Each Test Plan is marked as passed when all Test Steps pass or as failed if a single Test Step fails. Failures are addressed per SOP002 Change Management. This serves as a record of the completed test.

Test Plans are automatically run by the computer, generating a report in PDF format. This Report Package is reviewed prior to execution per SOP004 Software Devopment Procedure. The Report Package is routed and archived in the Quality Management System. When it becomes necessary to annotate a computer-generated document, SOP001 Good Documentation Practices must be followed.

1.13 Test Plan Storage and Review

This Test Plan is part of a Company's automated validation framework. The framework consists of the following parts:

LaTeX files are used to provide an Abstract, Introduction, Intended Use Requirements, Test Plan Overview, Test Equipment, Configuration Item Validation, Conclusion, and Change Summary. LaTeX files are converted and assembled into PDF documents. PDF documents are routed using the Company's document management system for approval.

Ruby software is used to run the automated framework to collect test evidence.

Git is used as the storage repository for LaTeX & YAML files, and a Git pull request is used to review the LaTeX & YAML files prior to use.

Evidence Test Plan output includes one Test Suite, one Test Plan, one Test Step, and Test Evidence.

YAML files define the Test Plan, Test Suite, and Test Steps that are processed to generate test evidence.

2 Requirements

Intended-use requirements are defined using the following story format:

As a <type of user>, I want <some goal> so that <some reason>

AMBER-IUR-001

As the designer, I want to demonstrate Amber can process Test Plans, Test Suites, and Test Cases so that I can produce a testing report.

AMBER-IUR-002

As the designer, I want to demonstrate Amber can invoke an another executable program so that I can collect evidence for my test reports.

AMBER-IUR-003

As the designer, I want to demonstrate Amber can accept command line options so that I can control the type of testing Amber conducts.

AMBER-IUR-004

As the designer, I want to demonstrate Amber can support nested Test Suites and Test Cases so that test objects can be logically organized.

AMBER-IUR-005

As the designer, I want to demonstrate Amber can substitute keywords when processing YAML files so that the maintenance of Test Plans, Test Suites, and Test Cases is minimized.

AMBER-IUR-006

As the designer, I want to demonstrate Amber can support embedded LaTeXenumerate and itemized commands so that Test Plans, Test Suites and Test Cases have beautifully typeset lists.

3 Test Plan Overview

This section describes Test Plans, Test Suites, Test Cases, and Test Steps that demonstrate how a Configuration Item satisfies the IUR. Each Test Plan describes any setup criteria needed to conduct the Test Steps. Each Test Plan contains a list of IURs and the Test Steps that demonstrate how the Configuration Item satisfies the IUR. Each Test Step is marked passed or failed as it is completed. Each Test Plan is marked as passed when all Test Steps pass or as failed if a single Test Step fails. This serves as a record of completed Test Plans and Test Steps.
Each Test Plan is described in its own section. The order the Test Plans are listed in is the order in which they are run. Each Test Plan defines:

name	Each Plan, Suite, and Case has a unique name.
purpose	Each Plan, Suite, and Case has a purpose.
Test Steps	Each step has a confirmation and expectation along with the command needed to challenge the IUR.
Objective Evidence	A record that the Test Plan was run along with any evidence collected while the Test Steps were run.
Traceability	Suites and Cases are traced to an IUR that is challenged. IUR can be traced to multiple Suites and Cases.

Each Test Plan, Test Suite, Test Case, and Test Step has been designed to be run by the computer. However, a person may choose to manually run the Test Step, save the test results, and generate this test report as specified in the appropriate design documentation. The example below runs these commands:

1. git help
2. cat .gitconfig

The output from both commands is written to the system console.

```
plan:
  name: A Test Plan Name
  purpose: purpose of the plan

suite:
  name: A Test Suite Name
  purpose: a suite purpose
  requirement: IUR01 and IUR02

  - case:
    name: A Test Case name
    purpose: A Test Case purpose
    steps:
      - confirm: Confirm git help is written to the console output.
        expectation: Git help is displayed.
        command: git
        argument: help

      - confirm: Confirm .git config is written to the console.
        expectation: .gitconfig is written to the console output.
        command: cat
        argument: .gitconfig
```

4 Test Evidence

The Company's automation framework assembles the content in this section. The section has one or more Test Plans, Test Suites, Test Cases, and Test Evidence. The evidence provided is used to conclude the Tool has met the Intended Use Requirements.

4.1 Test Plan: master

Purpose: LaTeX. LaTeX This Test Plan demonstrates the Ruby gem Amber functions correctly. This Test plan shows Amber has met the intended-use requirements defined by the designer. This Test Plan includes the Test Suites listed below.

- Test Suite cli/options shows all permutations of command line options function correctly.
- Test Suite utility/substitution demonstrates how Amber substitutes values found in Test Plans, Test Suites, and Test Cases to simplify test maintenance.
- Test Suite tif/structure reveals concepts unique to a Test Input Factory
- Test Suite advanced-concept demonstrates embedded LaTeX commands throughout.

Amber Test Cases have been designed to show off Amber concepts. There are many Test Steps that may report failure. In these cases, an operation system application is simply not installed. For example, most Linux systems come with the man program installed. Amber will issue a command 'which man' and report PASS if man is installed and FAIL if man is not installed. In the failed test case, Amber prints the environment path that was searched. In short, Amber simply ran the command it was instructed to run and captured the results. This is sufficient for Amber's validation purposes.

4.1.1 Test Suite: options

Purpose: This test suite demonstrates Amber consumes and uses command line arguments properly.

4.1.2 Test Case: browser

Purpose: This test case is used to demonstrate Amber properly uses the –browser command line option.

Requirement: AMBER-IUR-001, AMBER-IUR-002 and AMBER-IUR-003

Step: 1

Confirm: Amber properly consumes the command line argument –browser.

Expectation: RSpec output shows Amber::CommandLineOptions.browser_option handles supported argument formats.

Command: rspec –format documentation -e 'Amber CLO Browser'

Execution start: Jun 08, 2026 15:37:06.235076

Execution end: Jun 08, 2026 15:37:06.401542

Test Result: PASS

Evidence: Starts on next line.

```
Run options: include {full_description: /Amber\ CLO\ Browser/}

Amber CLO Browser
  --browser=Chrome
    has been used from the command line.
  --browser Chrome
    has been used from the command line.
  -bChrome
    has been used from the command line.
  --browser=Firefox
    has been used from the command line.
  --browser Firefox
    has been used from the command line.
  -bFirefox
    has been used from the command line.
  --browser=IE
    has been used from the command line.
  --browser IE
    has been used from the command line.
  -bIE
    has been used from the command line.
  --browser=Opra
    has been used from the command line.
  --browser Opra
    has been used from the command line.
  -bOpra
    has been used from the command line.

Finished in 0.009 seconds (files took 0.10591 seconds to load)
12 examples, 0 failures

Coverage report generated for Unit Tests to /home/traap/soup/amber/coverage.
Line Coverage: 60.93% (549 / 901)
```

4.1.3 Test Case: case

Purpose: This test case is used to demonstrate Amber properly uses the –case command line option.

Requirement: AMBER-IUR-001, AMBER-IUR-002 and AMBER-IUR-003

Step: 1

Confirm: Amber properly consumes the command line argument –case.

Expectation: RSpec output shows Amber::CommandLineOptions.case_option handles supported argument formats.

Command: rspec –format documentation -e 'Amber CLO Case'

Execution start: Jun 08, 2026 15:37:06.402315

Execution end: Jun 08, 2026 15:37:06.570062

Test Result: PASS

Evidence: Starts on next line.

```
Run options: include {full_description: /Amber\ CLO\ Case/}

Amber CLO Case
  -c has not been used.
  -cbar has been used from the command line.
  -c foobar has been used from the command line.

Amber CLO Case
  --case has not been used.
  --case=foo has been used from the command line.
  --case baz has been used from the command line.

Finished in 0.01047 seconds (files took 0.10777 seconds to load)
6 examples, 0 failures

Coverage report generated for Unit Tests to /home/traap/soup/amber/coverage.
Line Coverage: 59.38% (535 / 901)
```

4.1.4 Test Case: default

Purpose: This test case is used to demonstrate Amber properly constructs a default Amber::Options object.

Requirement: AMBER-IUR-001, AMBER-IUR-002 and AMBER-IUR-003

Step: 1

Confirm: Amber properly constructs a default Amber::Options object.

Expectation: RSpec output shows Amber::Options object is initialized correctly.

Command: rspec –format documentation -e 'Amber CLO Defaults'

Execution start: Jun 08, 2026 15:37:06.570907

Execution end: Jun 08, 2026 15:37:06.736012

Test Result: PASS

Evidence: Starts on next line.

```
Run options: include {full_description: /Amber\ CLO\ Defaults/}

All examples were filtered out

Finished in 0.00012 seconds (files took 0.10973 seconds to load)
0 examples, 0 failures

Coverage report generated for Unit Tests to /home/traap/soup/amber/coverage.
Line Coverage: 56.94% (513 / 901)
```

4.1.5 Test Case: environment

Purpose: This test case demonstrates that Amber can record the operational environment in which it was used.

Requirement: AMBER-IUR-001, AMBER-IUR-002 and AMBER-IUR-003

Step: 1

Confirm: Amber understands when to record the operational environment.

Expectation: RSpec output shows Amber's understanding of the environment option.

Command: rspec –format documentation -e 'Amber CLO Environment'

Execution start: Jun 08, 2026 15:37:06.736829

Execution end: Jun 08, 2026 15:37:06.913117

Test Result: PASS

Evidence: Starts on next line.

```
Run options: include {full_description: /Amber\ CLO\ Environment/}

Amber CLO Environment
  no -e
    has not been used.
  -e
    has been used from the command line.
  --environment
    has been used from the command line.

Amber CLO Environment
  dollar_signs are escaped.

Finished in 0.00937 seconds (files took 0.11414 seconds to load)
4 examples, 0 failures

Coverage report generated for Unit Tests to /home/traap/soup/amber/coverage.
Line Coverage: 58.6% (528 / 901)
```

4.1.6 Test Case: file

Purpose: This test case is used to demonstrate Amber properly uses the –file command line option.

Requirement: AMBER-IUR-001, AMBER-IUR-002 and AMBER-IUR-003

Step: 1

Confirm: Amber properly consumes the command line argument –file.

Expectation: RSpec output shows Amber::CommandLineOptions.file_option handles supported argument formats.

Command: rspec –format documentation -e 'Amber CLO File'

Execution start: Jun 08, 2026 15:37:06.914038

Execution end: Jun 08, 2026 15:37:07.079089

Test Result: PASS

Evidence: Starts on next line.

```
Run options: include {full_description: /Amber\ CLO\ File/}

Amber CLO File Short
  no -f has not been used.
  -fa.yaml has been used from the command line.

Amber CLO File Long
  --file has not been used.
  --file=b.yaml has been used from the command line.
  --file c.yaml has been used from the command line.

Finished in 0.00789 seconds (files took 0.10547 seconds to load)
5 examples, 0 failures

Coverage report generated for Unit Tests to /home/traap/soup/amber/coverage.
Line Coverage: 58.93% (531 / 901)
```

4.1.7 Test Case: language

Purpose: This test case is used to demonstrate Amber properly uses the –language command line option.

Requirement: AMBER-IUR-001, AMBER-IUR-002 and AMBER-IUR-003

Step: 1

Confirm: Amber properly consumes the command line argument –language.

Expectation: RSpec output shows Amber::CommandLineOptions.language_option handles supported argument formats.

Command: rspec –format documentation -e 'Amber CLO Language'

Execution start: Jun 08, 2026 15:37:07.079968

Execution end: Jun 08, 2026 15:37:07.259651

Test Result: PASS

Evidence: Starts on next line.

```
Run options: include {full_description: /Amber\ CLO\ Language/}

Amber CLO Language
  no -l
    has not been used.
  with unknown language
    --language XX
      raises an invalid argument exception
    --language=XX
      raises an invalid argument exception
    -l XX
      raises an invalid argument exception
    -lXX
      raises an invalid argument exception
  behaves like Amber CLO language parameter
    --language=zz
      returns n/a when run with double dash and equal sign
    --language zz
      returns n/a when run with double dash and a space
    -lzz
      returns n/a when run with dash and no space
    -l zz
      returns n/a when run with dash and a space
  behaves like Amber CLO language parameter
    --language=cs
      returns Czech when run with double dash and equal sign
    --language cs
      returns Czech when run with double dash and a space
    -lcs
      returns Czech when run with dash and no space
    -l cs
      returns Czech when run with dash and a space
  behaves like Amber CLO language parameter
    --language=cy
      returns Welsh when run with double dash and equal sign
    --language cy
      returns Welsh when run with double dash and a space
    -lcy
      returns Welsh when run with dash and no space
    -l cy
      returns Welsh when run with dash and a space
  behaves like Amber CLO language parameter
    --language=da
      returns Danish when run with double dash and equal sign
    --language da
      returns Danish when run with double dash and a space
    -lda
```

```
      returns Danish when run with dash and no space
  -l da
      returns Danish when run with dash and a space
behaves like Amber CLO language parameter
  --language=de
      returns German when run with double dash and equal sign
  --language de
      returns German when run with double dash and a space
  -lde
      returns German when run with dash and no space
  -l de
      returns German when run with dash and a space
behaves like Amber CLO language parameter
  --language=en
      returns English when run with double dash and equal sign
  --language en
      returns English when run with double dash and a space
  -len
      returns English when run with dash and no space
  -l en
      returns English when run with dash and a space
behaves like Amber CLO language parameter
  --language=es
      returns Spanish; Castilian when run with double dash and equal sign
  --language es
      returns Spanish; Castilian when run with double dash and a space
  -les
      returns Spanish; Castilian when run with dash and no space
  -l es
      returns Spanish; Castilian when run with dash and a space
behaves like Amber CLO language parameter
  --language=fi
      returns Finnish when run with double dash and equal sign
  --language fi
      returns Finnish when run with double dash and a space
  -lfi
      returns Finnish when run with dash and no space
  -l fi
      returns Finnish when run with dash and a space
behaves like Amber CLO language parameter
  --language=fr
      returns French when run with double dash and equal sign
  --language fr
      returns French when run with double dash and a space
  -lfr
      returns French when run with dash and no space
  -l fr
      returns French when run with dash and a space
behaves like Amber CLO language parameter
  --language=fr-ca
      returns CA French - Canadian when run with double dash and equal sign
  --language fr-ca
      returns CA French - Canadian when run with double dash and a space
  -lfr-ca
      returns CA French - Canadian when run with dash and no space
  -l fr-ca
      returns CA French - Canadian when run with dash and a space
behaves like Amber CLO language parameter
  --language=fr-eu
      returns EU French - European when run with double dash and equal sign
  --language fr-eu
      returns EU French - European when run with double dash and a space
  -lfr-eu
      returns EU French - European when run with dash and no space
  -l fr-eu
      returns EU French - European when run with dash and a space
behaves like Amber CLO language parameter
  --language=fy
      returns Western Frisian when run with double dash and equal sign
  --language fy
      returns Western Frisian when run with double dash and a space
  -lfy
      returns Western Frisian when run with dash and no space
```

```
  -l fy
    returns Western Frisian when run with dash and a space
behaves like Amber CLO language parameter
  --language=it
    returns Italian when run with double dash and equal sign
  --language it
    returns Italian when run with double dash and a space
  -lit
    returns Italian when run with dash and no space
  -l it
    returns Italian when run with dash and a space
behaves like Amber CLO language parameter
  --language=nl
    returns Dutch; Flemish when run with double dash and equal sign
  --language nl
    returns Dutch; Flemish when run with double dash and a space
  -lnl
    returns Dutch; Flemish when run with dash and no space
  -l nl
    returns Dutch; Flemish when run with dash and a space
behaves like Amber CLO language parameter
  --language=no
    returns Norwegian when run with double dash and equal sign
  --language no
    returns Norwegian when run with double dash and a space
  -lno
    returns Norwegian when run with dash and no space
  -l no
    returns Norwegian when run with dash and a space
behaves like Amber CLO language parameter
  --language=pl
    returns Polish when run with double dash and equal sign
  --language pl
    returns Polish when run with double dash and a space
  -lpl
    returns Polish when run with dash and no space
  -l pl
    returns Polish when run with dash and a space
behaves like Amber CLO language parameter
  --language=pt
    returns Portuguese when run with double dash and equal sign
  --language pt
    returns Portuguese when run with double dash and a space
  -lpt
    returns Portuguese when run with dash and no space
  -l pt
    returns Portuguese when run with dash and a space
behaves like Amber CLO language parameter
  --language=ro
    returns Romanian; Moldavian; Moldovan when run with double dash and equal sign
  --language ro
    returns Romanian; Moldavian; Moldovan when run with double dash and a space
  -lro
    returns Romanian; Moldavian; Moldovan when run with dash and no space
  -l ro
    returns Romanian; Moldavian; Moldovan when run with dash and a space
behaves like Amber CLO language parameter
  --language=ru
    returns Russian when run with double dash and equal sign
  --language ru
    returns Russian when run with double dash and a space
  -lru
    returns Russian when run with dash and no space
  -l ru
    returns Russian when run with dash and a space
behaves like Amber CLO language parameter
  --language=sk
    returns Slovak when run with double dash and equal sign
  --language sk
    returns Slovak when run with double dash and a space
  -lsk
    returns Slovak when run with dash and no space
  -l sk
```

```
      returns Slovak when run with dash and a space
  behaves like Amber CLO language parameter
    --language=sv
      returns Swedish when run with double dash and equal sign
    --language sv
      returns Swedish when run with double dash and a space
    -lsv
      returns Swedish when run with dash and no space
    -l sv
      returns Swedish when run with dash and a space
  behaves like Amber CLO language parameter
    --language=zu
      returns Zulu when run with double dash and equal sign
    --language zu
      returns Zulu when run with double dash and a space
    -lzu
      returns Zulu when run with dash and no space
    -l zu
      returns Zulu when run with dash and a space

Finished in 0.01499 seconds (files took 0.11225 seconds to load)
93 examples, 0 failures

Coverage report generated for Unit Tests to /home/traap/soup/amber/coverage.
Line Coverage: 60.38% (544 / 901)
```

4.1.8 Test Case: nodryrun

Purpose: This test case is used to demonstrate Amber properly uses the –nodryrun command line option.

Requirement: AMBER-IUR-001, AMBER-IUR-002 and AMBER-IUR-003

Step: 1

Confirm: Amber properly consumes the command line argument –nodryrun.

Expectation: RSpec output shows Amber::CommandLineOptions.nodryrun_option handles supported argument formats.

Command: rspec –format documentation -e 'Amber CLO NoDryRun'

Execution start: Jun 08, 2026 15:37:07.260522

Execution end: Jun 08, 2026 15:37:07.429486

Test Result: PASS

Evidence: Starts on next line.

```
Run options: include {full_description: /Amber\ CLO\ NoDryRun/}

Amber CLO NoDryRun
  no -n
    has not been used.
  -n
    has been used from the command line.
  --nodryrun
    has been used from the command line.

Finished in 0.00778 seconds (files took 0.10989 seconds to load)
3 examples, 0 failures

Coverage report generated for Unit Tests to /home/traap/soup/amber/coverage.
Line Coverage: 57.82% (521 / 901)
```

4.1.9 Test Case: obliterate

Purpose: This test case is used to demonstrate Amber properly uses the –obliterate command line option.

Requirement: AMBER-IUR-001, AMBER-IUR-002 and AMBER-IUR-003

Step: 1

Confirm: Amber properly consumes the command line argument –obliterate.

Expectation: RSpec output shows Amber::CommandLineOptions.obliterate_option handles supported argument formats.

Command: rspec –format documentation -e 'Amber CLO Obliterate'

Execution start: Jun 08, 2026 15:37:07.430168

Execution end: Jun 08, 2026 15:37:07.596940

Test Result: PASS

Evidence: Starts on next line.

```
Run options: include {full_description: /Amber\ CLO\ Obliterate/}

Amber CLO Obliterate
  no –O
    has not been used.
  –O
    has been used from the command line.
  ––obliterate
    has been used from the command line.

Finished in 0.0075 seconds (files took 0.10686 seconds to load)
3 examples, 0 failures

Coverage report generated for Unit Tests to /home/traap/soup/amber/coverage.
Line Coverage: 57.82% (521 / 901)
```

4.1.10 Test Case: plan

Purpose: This test case is used to demonstrate Amber properly uses the –plan command line option.

Requirement: AMBER-IUR-001, AMBER-IUR-002 and AMBER-IUR-003

Step: 1

Confirm: Amber properly consumes the command line argument –plan.

Expectation: RSpec output shows Amber::CommandLineOptions.plan_option handles supported argument formats.

Command: rspec –format documentation -e 'Amber CLO Plan'

Execution start: Jun 08, 2026 15:37:07.597590

Execution end: Jun 08, 2026 15:37:07.764089

Test Result: PASS

Evidence: Starts on next line.

```
Run options: include {full_description: /Amber\ CLO\ Plan/}

Amber CLO Plan Short
  -p has not been used.
  -pbar has been used from the command line.
  -p foobar has been used from the command line.

Amber CLO Plan Long
  --plan has not been used.
  --plan=foo has been used from the command line.
  --plan baz has been used from the command line.

Finished in 0.01063 seconds (files took 0.10562 seconds to load)
6 examples, 0 failures

Coverage report generated for Unit Tests to /home/traap/soup/amber/coverage.
Line Coverage: 59.38% (535 / 901)
```

4.1.11 Test Case: simulate

Purpose: This test case is used to demonstrate Amber properly uses the –simulate command line option.

Requirement: AMBER-IUR-001, AMBER-IUR-002 and AMBER-IUR-003

Step: 1

Confirm: Amber properly consumes the command line argument –simulate.

Expectation: RSpec output shows Amber::CommandLineOptions.simulate_option handles supported argument formats.

Command: rspec –format documentation -e 'Amber CLO Simulate'

Execution start: Jun 08, 2026 15:37:07.764904

Execution end: Jun 08, 2026 15:37:07.933716

Test Result: PASS

Evidence: Starts on next line.

```
Run options: include {full_description: /Amber\ CLO\ Simulate/}

Amber CLO Simulate
  no -S
    has not been used.
  -S
    has been used from the command line.
  --simulate
    has been used from the command line.

Finished in 0.00779 seconds (files took 0.10794 seconds to load)
3 examples, 0 failures

Coverage report generated for Unit Tests to /home/traap/soup/amber/coverage.
Line Coverage: 57.82% (521 / 901)
```

4.1.12 Test Case: suite

Purpose: This test case is used to demonstrate Amber properly uses the –suite command line option.

Requirement: AMBER-IUR-001, AMBER-IUR-002 and AMBER-IUR-003

Step: 1

Confirm: Amber properly consumes the command line argument –suite.

Expectation: RSpec output shows Amber::CommandLineOptions.suite_option handles supported argument formats.

Command: rspec –format documentation -e 'Amber CLO Suite'

Execution start: Jun 08, 2026 15:37:07.934647

Execution end: Jun 08, 2026 15:37:08.102211

Test Result: PASS

Evidence: Starts on next line.

```
Run options: include {full_description: /Amber\ CLO\ Suite/}

Amber CLO Suite Short
  -s has not been used.
  -sbar has been used from the command line.
  -s foobar has been used from the command line.

Amber CLO Suite Long
  --suite has not been used.
  --suite=foo has been used from the command line.
  --suite baz has been used from the command line.

Finished in 0.00872 seconds (files took 0.10832 seconds to load)
6 examples, 0 failures

Coverage report generated for Unit Tests to /home/traap/soup/amber/coverage.
Line Coverage: 59.38% (535 / 901)
```

4.1.13 Test Case: verbose

Purpose: This test case is used to demonstrate Amber properly uses the –verbose command line option.

Requirement: AMBER-IUR-001, AMBER-IUR-002 and AMBER-IUR-003

Step: 1

Confirm: Amber properly consumes the command line argument –verbose.

Expectation: RSpec output shows Amber::CommandLineOptions.verbose_option handles supported argument formats.

Command: rspec –format documentation -e 'Amber CLO Verbose'

Execution start: Jun 08, 2026 15:37:08.103683

Execution end: Jun 08, 2026 15:37:08.264587

Test Result: PASS

Evidence: Starts on next line.

```
Run options: include {full_description: /Amber\ CLO\ Verbose/}

Amber CLO Verbose
  no -v
    has not been used.
  -v
    has been used from the command line.
  --verbose
    has been used from the command line.

Finished in 0.00777 seconds (files took 0.10342 seconds to load)
3 examples, 0 failures

Coverage report generated for Unit Tests to /home/traap/soup/amber/coverage.
Line Coverage: 57.82% (521 / 901)
```

4.1.14 Test Case: version

Purpose: This test case is used to demonstrate Amber properly uses the –version command line option.

Requirement: AMBER-IUR-001, AMBER-IUR-002 and AMBER-IUR-003

Step: 1

Confirm: Amber properly consumes the command line argument –version.

Expectation: RSpec output shows Amber::CommandLineOptions.version_option handles supported argument formats.

Command: rspec –format documentation -e 'Amber CLO Version'

Execution start: Jun 08, 2026 15:37:08.266411

Execution end: Jun 08, 2026 15:37:08.433853

Test Result: PASS

Evidence: Starts on next line.

```
Run options: include {full_description: /Amber\ CLO\ Version/}

Amber CLO Version
  no --version
    was not used. However the version number must match 1.6.4.414
  --version
1.6.4.414
    has been used from the command line.
  Version
    has a version number
    version number must match 1.6.4.414

Finished in 0.00852 seconds (files took 0.10767 seconds to load)
4 examples, 0 failures

Coverage report generated for Unit Tests to /home/traap/soup/amber/coverage.
Line Coverage: 57.71% (520 / 901)
```

4.1.15 Test Case: writer

Purpose: This test case is used to demonstrate Amber properly uses the –writer command line option.

Requirement: AMBER-IUR-001, AMBER-IUR-002 and AMBER-IUR-003

Step: 1

Confirm: Amber properly consumes the command line argument –writer.

Expectation: RSpec output shows Amber::CommandLineOptions.writer_option handles supported argument formats.

Command: rspec –format documentation -e 'Amber CLO Writer'

Execution start: Jun 08, 2026 15:37:08.436164

Execution end: Jun 08, 2026 15:37:08.599744

Test Result: PASS

Evidence: Starts on next line.

```
Run options: include {full_description: /Amber\ CLO\ Writer/}

Amber CLO Writer Short
  -w  has not been used.
  -wAscii has been used from the command line.
  -w Ascii has been used from the command line.
  -wLaTeX has been used from the command line.
  -w LaTeX has been used from the command line.

Amber CLO Writer Long
  --writer has not been used.
  --writer=Ascii has been used from the command line.
  --writer Ascii has been used from the command line.
  --writer=LaTeX has been used from the command line.
  --writer LaTeX has been used from the command line.

Finished in 0.00844 seconds (files took 0.10459 seconds to load)
10 examples, 0 failures

Coverage report generated for Unit Tests to /home/traap/soup/amber/coverage.
Line Coverage: 60.04% (541 / 901)
```

4.1.16 Test Suite: substitute

Purpose: This test suite demonstrates Amber's runtime substitution capabilities. Amber has been designed to translate the keywords below.

1. browser or BROWSER
2. file or FILE
3. language or LANGUAGE
4. language-code or LANGUAGE-CODE
5. home or HOME or ~

This Test case also demonstrated embedded LaTeX syntax to define the list above.

4.1.17 Test Case: browser

Purpose: This test case is used to demonstrate the browser keyword is properly substituted by Amber.

Requirement: AMBER-IUR-004 and AMBER-IUR-005

Step: 1

Confirm: Amber properly substitutes the browser keyword for all browser types.

Expectation: RSpec output shows Amber::Substitute.browser properly substituted browser and BROWSER keywords to Chrome, Firefox, Edge, and IE.

Command: rspec –format documentation -e 'YAML Browser Substitutions'

Execution start: Jun 08, 2026 15:37:08.699570

Execution end: Jun 08, 2026 15:37:08.862917

Test Result: PASS

Evidence: Starts on next line.

```
Run options: include {full_description: /YAML\ Browser\ Substitutions/}

YAML Browser Substitutions
  Amber::Substitute.browser
    can substitute ${BROWSER} to None
    can substitute ${browser} to None

YAML Browser Substitutions
  Amber::Substitute.browser
    can substitute ${BROWSER} to Brave

YAML Browser Substitutions
  Amber::Substitute.browser
    can substitute ${BROWSER} to Chrome

YAML Browser Substitutions
  Amber::Substitute.browser
    can substitute ${browser} to Edge

YAML Browser Substitutions
  Amber::Substitute.browser
    can substitute ${BROWSER} to Firefox

YAML Browser Substitutions
  Amber::Substitute.browser
    can substitute ${browser} to IE

YAML Browser Substitutions
  Amber::Substitute.browser
    can substitute ${BROWSER} to Opera
```

```
31
32 Finished in 0.00853 seconds (files took 0.10544 seconds to load)
33 8 examples, 0 failures
34
35 Coverage report generated for Unit Tests to /home/traap/soup/amber/coverage.
36 Line Coverage: 59.38% (535 / 901)
```

4.1.18 Test Case: extend-path

Purpose: This test case is used to demonstrate the tilde marker is properly substituted by Amber.

Requirement: AMBER-IUR-004 and AMBER-IUR-005

Step: 1

Confirm: Amber properly substitutes the tilde marker correctly for the operating system.

Expectation: RSpec output shows Amber::Substitute.extend_path substituted ~ to the home directory.

Command: rspec –format documentation -e 'YAML Extend Path Substitutions'

Execution start: Jun 08, 2026 15:37:08.863579

Execution end: Jun 08, 2026 15:37:09.033170

Test Result: PASS

Evidence: Starts on next line.

```
Run options: include {full_description: /YAML\ Extend\ Path\ Substitutions/}

YAML Extend Path Substitutions
  Amber::Substitute.expected_path
    can expand ~ to /home/traap
    can expand ~ and ~ to /home/traap and /home/traap

Finished in 0.00824 seconds (files took 0.11012 seconds to load)
2 examples, 0 failures

Coverage report generated for Unit Tests to /home/traap/soup/amber/coverage.
Line Coverage: 57.16% (515 / 901)
```

4.1.19 Test Case: home

Purpose: This test case is used to demonstrate the home keyword is properly substituted by Amber.

Requirement: AMBER-IUR-004 and AMBER-IUR-005

Step: 1

Confirm: Amber properly substitutes the home keyword for all browser types.

Expectation: RSpec output shows Amber::Substitute.home properly substituted home and HOME keywords specific to this operating system.

Command: rspec –format documentation -e 'YAML Home Substitutions'

Execution start: Jun 08, 2026 15:37:09.033996

Execution end: Jun 08, 2026 15:37:09.198011

Test Result: PASS

Evidence: Starts on next line.

```
Run options: include {full_description: /YAML\ Home\ Substitutions/}

YAML Home Substitutions
  Amber::Substitute.home
    can substitute ${home} to ~
    can substitute ${HOME} to ~
    can substitute ${home} and ${HOME} to ~ and ~

Finished in 0.00815 seconds (files took 0.10627 seconds to load)
3 examples, 0 failures

Coverage report generated for Unit Tests to /home/traap/soup/amber/coverage.
Line Coverage: 57.38% (517 / 901)
```

4.1.20 Test Case: language

Purpose: This test case is used to demonstrate the language keyword is properly substituted by Amber.

Requirement: AMBER-IUR-004 and AMBER-IUR-005

Step: 1

Confirm: Amber properly substitutes the language keyword for all supported languages.

Expectation: RSpec output shows Amber::Substitute.language properly substituted language and LANGUAGE keywords to zz, cs, da, de, en, es, fr-ca, fr-eu, it, ne, no, pl, so, and sv.

Command: rspec –format documentation -e 'YAML Language Substitutions'

Execution start: Jun 08, 2026 15:37:09.198691

Execution end: Jun 08, 2026 15:37:09.364954

Test Result: PASS

Evidence: Starts on next line.

```
Run options: include {full_description: /YAML\ Language\ Substitutions/}

YAML Language Substitutions
  behaves like Amber::Substitute.language
    can substitute ${language} to n/a
    can substitute ${LANGUAGE} to n/a
  behaves like Amber::Substitute.language
    can substitute ${language} to Czech
    can substitute ${LANGUAGE} to Czech
  behaves like Amber::Substitute.language
    can substitute ${language} to Welsh
    can substitute ${LANGUAGE} to Welsh
  behaves like Amber::Substitute.language
    can substitute ${language} to Danish
    can substitute ${LANGUAGE} to Danish
  behaves like Amber::Substitute.language
    can substitute ${language} to German
    can substitute ${LANGUAGE} to German
  behaves like Amber::Substitute.language
    can substitute ${language} to English
    can substitute ${LANGUAGE} to English
  behaves like Amber::Substitute.language
    can substitute ${language} to Spanish; Castilian
    can substitute ${LANGUAGE} to Spanish; Castilian
  behaves like Amber::Substitute.language
    can substitute ${language} to Finnish
    can substitute ${LANGUAGE} to Finnish
  behaves like Amber::Substitute.language
    can substitute ${language} to French
    can substitute ${LANGUAGE} to French
  behaves like Amber::Substitute.language
    can substitute ${language} to CA French – Canadian
    can substitute ${LANGUAGE} to CA French – Canadian
  behaves like Amber::Substitute.language
    can substitute ${language} to EU French – European
    can substitute ${LANGUAGE} to EU French – European
  behaves like Amber::Substitute.language
    can substitute ${language} to Western Frisian
    can substitute ${LANGUAGE} to Western Frisian
  behaves like Amber::Substitute.language
    can substitute ${language} to Italian
    can substitute ${LANGUAGE} to Italian
  behaves like Amber::Substitute.language
    can substitute ${language} to Dutch; Flemish
    can substitute ${LANGUAGE} to Dutch; Flemish
  behaves like Amber::Substitute.language
```

```
    can substitute ${language} to Norwegian
    can substitute ${LANGUAGE} to Norwegian
  behaves like Amber::Substitute.language
    can substitute ${language} to Polish
    can substitute ${LANGUAGE} to Polish
  behaves like Amber::Substitute.language
    can substitute ${language} to Portuguese
    can substitute ${LANGUAGE} to Portuguese
  behaves like Amber::Substitute.language
    can substitute ${language} to Romanian; Moldavian; Moldovan
    can substitute ${LANGUAGE} to Romanian; Moldavian; Moldovan
  behaves like Amber::Substitute.language
    can substitute ${language} to Russian
    can substitute ${LANGUAGE} to Russian
  behaves like Amber::Substitute.language
    can substitute ${language} to Slovak
    can substitute ${LANGUAGE} to Slovak
  behaves like Amber::Substitute.language
    can substitute ${language} to Swedish
    can substitute ${LANGUAGE} to Swedish
  behaves like Amber::Substitute.language
    can substitute ${language} to Zulu
    can substitute ${LANGUAGE} to Zulu

Finished in 0.00889 seconds (files took 0.10649 seconds to load)
44 examples, 0 failures

Coverage report generated for Unit Tests to /home/traap/soup/amber/coverage.
Line Coverage: 57.16% (515 / 901)
```

4.1.21 Test Case: language-code

Purpose: This test case is used to demonstrate the language-code keyword is properly substituted by Amber.

Requirement: AMBER-IUR-004 and AMBER-IUR-005

Step: 1

Confirm: Amber properly substitutes the language-code keyword for all supported languages.

Expectation: RSpec output shows Amber::Substitute.language-code properly substituted language-code and LANGUAGE-CODE keywords to n/a, Czech, Dansk, Deutsch, English, Espanol, CA French - Canadian, EU French - European, Italiano, Nederlands, Norsk, Polish, Romanian, and Svenska.

Command: rspec –format documentation -e 'YAML Language Code Substitutions'

Execution start: Jun 08, 2026 15:37:09.365668

Execution end: Jun 08, 2026 15:37:09.529090

Test Result: PASS

Evidence: Starts on next line.

```
Run options: include {full_description: /YAML\ Language\ Code\ Substitutions/}

YAML Language Code Substitutions
  behaves like Amber::Substitute.language_code
    substitutes ${LANGUAGE-CODE} to zz
    substitutes ${language-code} to zz
  behaves like Amber::Substitute.language_code
    substitutes ${LANGUAGE-CODE} to cs
    substitutes ${language-code} to cs
  behaves like Amber::Substitute.language_code
    substitutes ${LANGUAGE-CODE} to cy
    substitutes ${language-code} to cy
  behaves like Amber::Substitute.language_code
    substitutes ${LANGUAGE-CODE} to da
    substitutes ${language-code} to da
  behaves like Amber::Substitute.language_code
    substitutes ${LANGUAGE-CODE} to de
    substitutes ${language-code} to de
  behaves like Amber::Substitute.language_code
    substitutes ${LANGUAGE-CODE} to en
    substitutes ${language-code} to en
  behaves like Amber::Substitute.language_code
    substitutes ${LANGUAGE-CODE} to es
    substitutes ${language-code} to es
  behaves like Amber::Substitute.language_code
    substitutes ${LANGUAGE-CODE} to fi
    substitutes ${language-code} to fi
  behaves like Amber::Substitute.language_code
    substitutes ${LANGUAGE-CODE} to fr
    substitutes ${language-code} to fr
  behaves like Amber::Substitute.language_code
    substitutes ${LANGUAGE-CODE} to fr-ca
    substitutes ${language-code} to fr-ca
  behaves like Amber::Substitute.language_code
    substitutes ${LANGUAGE-CODE} to fr-eu
    substitutes ${language-code} to fr-eu
  behaves like Amber::Substitute.language_code
    substitutes ${LANGUAGE-CODE} to fy
    substitutes ${language-code} to fy
  behaves like Amber::Substitute.language_code
    substitutes ${LANGUAGE-CODE} to it
    substitutes ${language-code} to it
  behaves like Amber::Substitute.language_code
    substitutes ${LANGUAGE-CODE} to nl
```

```
    substitutes ${language_code} to nl
  behaves like Amber::Substitute.language_code
    substitutes ${LANGUAGE_CODE} to no
    substitutes ${language_code} to no
  behaves like Amber::Substitute.language_code
    substitutes ${LANGUAGE_CODE} to pl
    substitutes ${language_code} to pl
  behaves like Amber::Substitute.language_code
    substitutes ${LANGUAGE_CODE} to pt
    substitutes ${language_code} to pt
  behaves like Amber::Substitute.language_code
    substitutes ${LANGUAGE_CODE} to ro
    substitutes ${language_code} to ro
  behaves like Amber::Substitute.language_code
    substitutes ${LANGUAGE_CODE} to ru
    substitutes ${language_code} to ru
  behaves like Amber::Substitute.language_code
    substitutes ${LANGUAGE_CODE} to sk
    substitutes ${language_code} to sk
  behaves like Amber::Substitute.language_code
    substitutes ${LANGUAGE_CODE} to sv
    substitutes ${language_code} to sv
  behaves like Amber::Substitute.language_code
    substitutes ${LANGUAGE_CODE} to zu
    substitutes ${language_code} to zu

Finished in 0.00945 seconds (files took 0.10461 seconds to load)
44 examples, 0 failures

Coverage report generated for Unit Tests to /home/traap/soup/amber/coverage.
Line Coverage: 57.16% (515 / 901)
```

4.1.22 Test Case: strings

Purpose: This test case is used to demonstrate the Amber substitutes multiple keywords in data stream.

Requirement: AMBER-IUR-004 and AMBER-IUR-005

Step: 1

Confirm: Amber properly substitutes all keywords in a data stream.

Expectation: RSpec output shows Amber::Substitute.strings substituted all keywords without encountering an error.

Command: rspec –format documentation -e 'YAML Strings Substitutions'

Execution start: Jun 08, 2026 15:37:09.529797

Execution end: Jun 08, 2026 15:37:09.693919

Test Result: PASS

Evidence: Starts on next line.

```
Run options: include {full_description: /YAML\ Strings\ Substitutions/}

YAML Strings Substitutions
  Amber::Substitute.strings
    can substitute ${BROWSER} to Opera
    can substitute ${browser} ${file} ${language} and ${language-code} to Opera baz Swedish and sv
    can substitute ${language-code}${file}${language}${browser} to svbazSwedishOpera

Finished in 0.00868 seconds (files took 0.10387 seconds to load)
3 examples, 0 failures

Coverage report generated for Unit Tests to /home/traap/soup/amber/coverage.
Line Coverage: 57.27% (516 / 901)
```

4.1.23 Test Suite: structure

Purpose: This test suite demonstrated Amber's ability to locate a nested Test Plan, Test Suite, or Test Case YAML file.

4.1.24 Test Case: factory

Purpose: This test case is used to demonstrate Amber can properly locate a nested Test Plan, Test Suite, or Test Case.

Requirement: AMBER-IUR-004

Step: 1

Confirm: Amber properly locates nested Test Plan, Test Suite, and Test Case names.

Expectation: RSpec output shows Amber::FactoryStructure properly locates the YAML file below.

1. Test Plan foo
2. Nested Test Plan baz
3. Test Suite foo
4. Nested Test Suite name baz
5. Test Case foo
6. Nested Test Case name baz

Command: rspec –format documentation -e 'Factory Structure'

Execution start: Jun 08, 2026 15:37:09.801091

Execution end: Jun 08, 2026 15:37:09.969277

Test Result: PASS

Evidence: Starts on next line.

```
Run options: include {full_description: /Factory\ Structure/}

Factory Structure
  Test Plan Name
    is normal.
    is nested.

Factory Structure
  Test Suite Name
    is normal.
    is nested.

Factory Structure
  Test Case Name
    is normal.
    is nested.

Finished in 0.00818 seconds (files took 0.10991 seconds to load)
6 examples, 0 failures

Coverage report generated for Unit Tests to /home/traap/soup/amber/coverage.
Line Coverage: 58.27% (525 / 901)
```

4.1.25 Test Suite: advanced-concept

Purpose: This Test Suite demonstrates a future concept that might be implemented. The concepts include the items below.

1. setup-before-all are Test Steps that are run before all Test Cases.
2. setup-before-each are Test Steps that are run before each Test Case.
3. teardown-after-all are Test Steps that are run before all Test Cases.
4. teardown-after-each are Test Steps that are run before each Test Case.

This Test Suite does demonstrate embedded LaTeX commands and it includes the following Test Cases.

- t001
- t002
- t003
- t005
- t006

Requirement: AMBER-IUR-006

4.1.26 Test Case: t001

Purpose: This Test Case demonstrates embedded LaTeX enumerate and itemized commands.

1. Step #1 uses the Linux echo command.
2. Step #2 will use Linux date command.
3. Step #3 confirms the Linux curl program is installed.

Requirement: AMBER-IUR-006

Step: 1

Confirm:
- Program echo has been installed.

Expectation:
- echo installation location is displayed.

Command: sudo which echo

Execution start: Jun 08, 2026 15:37:10.068954

Execution end: Jun 08, 2026 15:37:10.076856

Test Result: PASS

Evidence: Starts on next line.

```
1 /usr/bin/echo
```

Step: 2

Confirm:
- Program date has been installed.

Expectation:
- date installation location is displayed.

Command: which date

Execution start: Jun 08, 2026 15:37:10.077018

Execution end: Jun 08, 2026 15:37:10.077384

Test Result: PASS

Evidence: Starts on next line.

```
/usr/bin/date
```

Step: 3

Confirm:
- Program curl has been installed.

Expectation:
- curl installation location is displayed.

Command: which curl

Execution start: Jun 08, 2026 15:37:10.077593

Execution end: Jun 08, 2026 15:37:10.078106

Test Result: PASS

Evidence: Starts on next line.

```
/usr/bin/curl
```

4.1.27 Test Case: t002

Purpose: This Test Case demonstrates embedded LaTeX enumerate command.

1. Step #1 grep check
2. Step #2 openssl check
3. Step #3 sed check
4. Step #4 tr check

Requirement: AMBER-IUR-006

Step: 1

Confirm: Program grep has been installed.

Expectation: grep installation location is displayed.

Command: sudo which grep

Execution start: Jun 08, 2026 15:37:10.078440

Execution end: Jun 08, 2026 15:37:10.084787

Test Result: PASS

Evidence: Starts on next line.

```
1 /usr/bin/grep
```

Step: 2

Confirm: Program openssl has been installed.

Expectation: openssl installation location is displayed.

Command: which openssl

Execution start: Jun 08, 2026 15:37:10.084922

Execution end: Jun 08, 2026 15:37:10.085210

Test Result: PASS

Evidence: Starts on next line.

```
1 /usr/bin/openssl
```

Step: 3

Confirm: Program sed has been installed.

Expectation: sed installation location is displayed.

Command: which sed

Execution start: Jun 08, 2026 15:37:10.085307

Execution end: Jun 08, 2026 15:37:10.086816

Test Result: PASS

Evidence: Starts on next line.

```
1 /usr/bin/sed
```

Step: 4

Confirm: Program tr has been installed.

Expectation: tr installation location is displayed.

Command: which tr

Execution start: Jun 08, 2026 15:37:10.086965

Execution end: Jun 08, 2026 15:37:10.087285

Test Result: PASS

Evidence: Starts on next line.

```
1 /usr/bin/tr
```

4.1.28 Test Case: t003

Purpose: This Test Case demonstrates embedded LaTeX enumerate command.

1. Step #1 nice check
2. Step #2 nl check
3. Step #3 bzmore check
4. Step #4 latexmk check
5. Step #5 git check

Requirement: AMBER-IUR-006

Step: 1

Confirm: Program nice has installed.

Expectation: nice installation location is displayed.

Command: sudo which nice

Execution start: Jun 08, 2026 15:37:10.087657

Execution end: Jun 08, 2026 15:37:10.093564

Test Result: PASS

Evidence: Starts on next line.

```
1 /usr/bin/nice
```

Step: 2

Confirm: Program nl has installed.

Expectation: nl installation location is displayed.

Command: which nl

Execution start: Jun 08, 2026 15:37:10.093698

Execution end: Jun 08, 2026 15:37:10.093964

Test Result: PASS

Evidence: Starts on next line.

```
1 /usr/bin/nl
```

Step: 3

Confirm: Program which bzmore has been installed.

Expectation: bzmore installation location is displayed.

Command: which bzmore

Execution start: Jun 08, 2026 15:37:10.094060

Execution end: Jun 08, 2026 15:37:10.094411

Test Result: PASS

Evidence: Starts on next line.

```
1 /usr/bin/bzmore
```

Step: 4

Confirm: Program latexmk has been installed.

Expectation: latexmk installation location is displayed.

Command: which latexmk

Execution start: Jun 08, 2026 15:37:10.094496

Execution end: Jun 08, 2026 15:37:10.095015

Test Result: PASS

Evidence: Starts on next line.

```
1 /usr/bin/latexmk
```

Step: 5

Confirm: A developer is able to access Git help.

Expectation: Git help is displayed.

Command: git help

Execution start: Jun 08, 2026 15:37:10.095557

Execution end: Jun 08, 2026 15:37:10.096534

Test Result: PASS

Evidence: Starts on next line.

```
usage: git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
           [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
           [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--no-lazy-fetch]
           [--no-optional-locks] [--no-advice] [--bare] [--git-dir=<path>]
           [--work-tree=<path>] [--namespace=<name>] [--config-env=<name>=<envvar>]
           <command> [<args>]

These are common Git commands used in various situations:

start a working area (see also: git help tutorial)
   clone      Clone a repository into a new directory
   init       Create an empty Git repository or reinitialize an existing one

work on the current change (see also: git help everyday)
   add        Add file contents to the index
   mv         Move or rename a file, a directory, or a symlink
   restore    Restore working tree files
   rm         Remove files from the working tree and from the index

examine the history and state (see also: git help revisions)
   bisect     Use binary search to find the commit that introduced a bug
   diff       Show changes between commits, commit and working tree, etc
   grep       Print lines matching a pattern
   log        Show commit logs
   show       Show various types of objects
   status     Show the working tree status

grow, mark and tweak your common history
   backfill   Download missing objects in a partial clone
   branch     List, create, or delete branches
   commit     Record changes to the repository
   history    EXPERIMENTAL: Rewrite history
   merge      Join two or more development histories together
   rebase     Reapply commits on top of another base tip
   reset      Set 'HEAD' or the index to a known state
   switch     Switch branches
   tag        Create, list, delete or verify tags

collaborate (see also: git help workflows)
   fetch      Download objects and refs from another repository
   pull       Fetch from and integrate with another repository or a local branch
   push       Update remote refs along with associated objects

'git help -a' and 'git help -g' list available subcommands and some
concept guides. See 'git help <command>' or 'git help <concept>'
to read about a specific subcommand or concept.
See 'git help git' for an overview of the system.
```

4.1.29 Test Case: t005

Purpose: This Test Case demonstrates embedded LaTeX enumerate command.

1. Step #1 bzless check
2. Step #2 zip check

Requirement: AMBER-IUR-006

Step: 1

Confirm: Program bzless has been installed.

Expectation: bzless installation location is displayed.

Command: which bzless

Execution start: Jun 08, 2026 15:37:10.097104

Execution end: Jun 08, 2026 15:37:10.097534

Test Result: FAIL

Evidence: Starts on next line.

```
1 which: no bzless in (/home/traap/.local/share/mise/installs/node/25.2.1/bin:/home/traap/.local
    /share/mise/installs/ruby/4.0.0/bin:/home/traap/.local/share/mise/shims:/home/traap/.local
    /share/omarchy/bin:/usr/local/sbin:/usr/local/bin:/usr/bin:/usr/lib/jvm/default/bin:/usr/
    bin/site_perl:/usr/bin/vendor_perl:/usr/bin/core_perl:/bin:/home/traap/.dotnet:/home/traap
    /.dotnet/tools:/usr/sbin:/home/traap/.local/bin:/home/traap/git/dotfiles/bash/bin:/home/
    traap/.fzf/bin:/home/traap/.dotnet:/home/traap/.dotnet/tools)
```

Step: 2

Confirm: Program zip has been installed.

Expectation: zip installation location is displayed.

Command: which zip

Execution start: Jun 08, 2026 15:37:10.097919

Execution end: Jun 08, 2026 15:37:10.098319

Test Result: PASS

Evidence: Starts on next line.

```
/usr/bin/zip
```

4.1.30 Test Case: t006

Purpose: This Test Case demonstrates embedded LaTeX enumerate command.

1. Step #1 curl check
2. Step #2 xxd check

Requirement: AMBER-IUR-006

Step: 1

Confirm: Program curl has been installed.

Expectation: curl installation location is displayed.

Command: which curl

Execution start: Jun 08, 2026 15:37:10.099020

Execution end: Jun 08, 2026 15:37:10.099571

Test Result: PASS

Evidence: Starts on next line.

```
1 /usr/bin/curl
```

Step: 2

Confirm: Program xxd has been installed.

Expectation: xxd installation location is displayed.

Command: which xxd

Execution start: Jun 08, 2026 15:37:10.100152

Execution end: Jun 08, 2026 15:37:10.100620

Test Result: PASS

Evidence: Starts on next line.

```
/usr/bin/xxd
```

4.2 System Environment

The hyphen character replaced the underscore character, and the forward slash character replaced the backslash character throughout this section.

ALLUSERSPROFILE:

APPDATA:

CLASSPATH: See below.

1. nil

COMPUTERNAME:

COMSPEC:

FP-NO-HOST-CHECK:

GIT-BRANCH:

HOME: /home/traap

HOMEDRIVE:

HOMEPATH: See below.

1. nil

HOSTNAME:

JRE-HOME:

LANG: en-US.UTF-8

LOCALAPPDATA:

LOGONSERVER:

NUMBER-OF-PROCESSORS:

OLDPWD: /home/traap/soup/amber

OS:

PATH: See below.

1. /home/traap/.local/share/mise/installs/node/25.2.1/bin
2. /home/traap/.local/share/mise/installs/ruby/4.0.0/bin
3. /home/traap/.local/share/mise/shims
4. /home/traap/.local/share/omarchy/bin
5. /usr/local/sbin
6. /usr/local/bin
7. /usr/bin
8. /usr/lib/jvm/default/bin
9. /usr/bin/site-perl
10. /usr/bin/vendor-perl
11. /usr/bin/core-perl
12. /bin
13. /home/traap/.dotnet
14. /home/traap/.dotnet/tools
15. /usr/sbin

16. /home/traap/.local/bin
17. /home/traap/git/dotfiles/bash/bin
18. /home/traap/.fzf/bin
19. /home/traap/.dotnet
20. /home/traap/.dotnet/tools

PRINTER:

PROCESSOR-ARCHITECTURE:

PROCESSOR-IDENTIFIER:

PROCESSOR-LEVEL:

PROCESSOR-REVISION:

PROFILEREAD:

ProgramData:

PROGRAMFILES:

ProgramFiles(x86):

ProgramW6432:

PSModulePath: See below.

1. nil

PUBLIC:

PWD: /home/traap/soup/amber

SESSIONNAME:

SHELL: /usr/bin/bash

SHLVL: 1

SYSTEMDRIVE:

SYSTEMROOT:

TEMP:

TMP:

TZ:

USER: traap

USERDNSDOMAIN:

USERDOMAIN:

USERNAME:

USERPROFILE:

WINDIR:

5 Configuration Item Conclusion

This Report Package demonstrates that the identified Configuration Item satisfies the Intended Use Requirements (IUR) referenced in this record and is considered validated for its intended use. This conclusion is based on the executed test reports, retained test evidence, reviewed results, and disposition of deviations or anomalies.

Change Summary

Change	Justification
[A] - Initial version.	New document.
[B] - Section 1 changes.	Reference gSOP.
[C] - Global report changes.	Amber is compliant with autodoc rolling releases.

Acronyms

Amce Corporation (COMPANY)

Amce Corporation is the organization responsible for the documents assembled with Autodoc.

DHF

820.3(e) (DFH) means any compilation of records which describe the design history of a finished device.